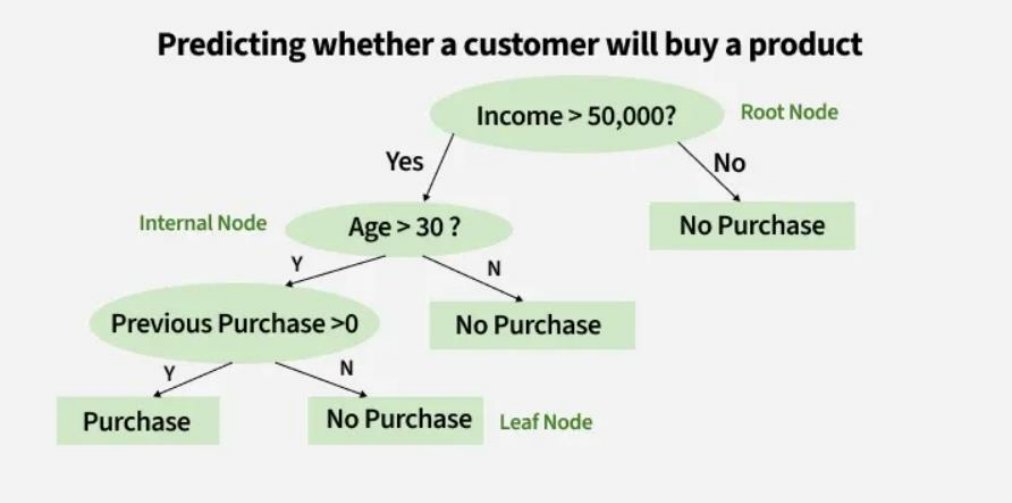
Week 2: Core Classification Models & Data Preprocessing

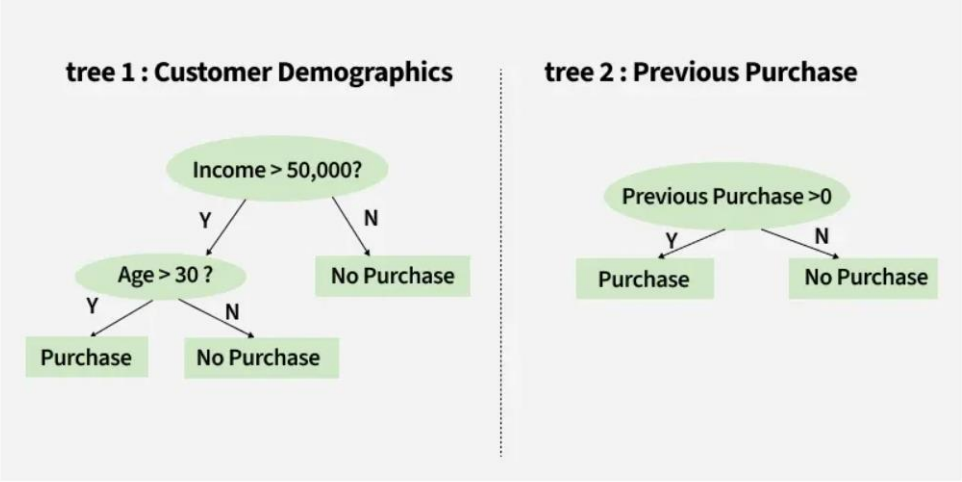
Decision tree:

A decision tree is a supervised learning algorithm used for both classification and regression tasks. It has a hierarchical tree structure which consists of a root node, branches, internal nodes and leaf nodes. It works like a flowchart that helps in making step-by-step decisions, where:

- Internal nodes represent attribute tests
- Branches represent attribute values
- Leaf nodes represent final decisions or predictions.



- If the person is above 30 and has not made previous purchases, predict "No Purchase" (leaf node).



Decision making with 2 Decision Tree

How Decision Trees Work

Learn these steps:

1. Load the dataset.
2. Select features and target.
3. Calculate impurity (Gini or Entropy).
4. Find the best feature for splitting.
5. Divide the dataset into smaller groups.
6. Repeat splitting recursively.
7. Stop when a stopping condition is reached.
8. Make the final prediction at the leaf node.

Pruning Techniques

Pruning means removing unnecessary branches from a Decision Tree to make it simpler and reduce overfitting.

Two main types:

- Pre-Pruning: Stops the tree from growing too much.
- Post-Pruning: First grows the tree, then removes unnecessary branches.

Hyperparameter Tuning

Hyperparameters control how a Decision Tree learns.

Important topics:

- criterion: Selects Gini, Entropy, or Log Loss for classification.
- max_depth: Controls tree depth.
- min_samples_split: Controls when a node can split.
- min_samples_leaf: Controls minimum samples in leaf nodes.
- max_features: Controls the number of features considered for splitting.
- max_leaf_nodes: Controls the number of leaf nodes.
- Grid Search: Tests all specified parameter combinations.
- Random Search: Tests randomly selected parameter combinations.

- Cross-Validation: Evaluates model performance across different data splits

Model Evaluation

After training a Decision Tree, we evaluate how well it performs.

For Classification:

Metric	Purpose
Accuracy	Percentage of correct predictions.
Precision	How many predicted positives were actually positive.
Recall	How many actual positives were correctly identified.
F1-Score	Balance between precision and recall.
Confusion Matrix	Shows correct and incorrect predictions.

For Regression:

- MAE (Mean Absolute Error)
- MSE (Mean Squared Error)
- RMSE (Root Mean Squared Error)
- R^2 Score

What is Bagging?

Definition: Bagging (Bootstrap Aggregating) is an ensemble learning technique that trains multiple models on different random samples of the training dataset and combines their predictions.

Instead of relying on one Decision Tree, we train many Decision Trees and combine their answers to get a more reliable prediction.

How does Bagging work?

1. Start with one training dataset.
2. Create multiple random samples using Bootstrap Sampling (sampling with replacement).
3. Train a separate Decision Tree on each sample.
4. Each tree makes its own prediction.
5. Combine the predictions using:
    - Majority Voting: For classification.

- Averaging: For regression.

Relationship Between Decision Tree, Bagging and Random Forest

Concept	Meaning
Decision Tree	A single tree-based ML model.
Bagging	Combines multiple models trained on different bootstrap samples.
Random Forest	Uses Bagging with Decision Trees and additional random feature selection.

Boosting:

Boosting is an Ensemble Learning technique that combines multiple weak learners (usually Decision Trees) sequentially to create a stronger and more accurate model.

Boosting trains multiple models one after another. Each new model tries to correct the mistakes made by the previous model.

Bagging vs Boosting

This is a very important interview concept.

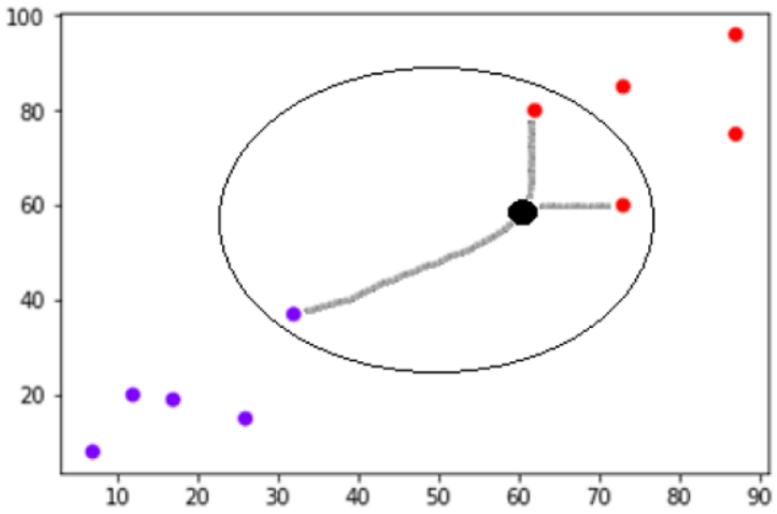
Bagging	Boosting
Models train independently.	Models train sequentially.
Uses different bootstrap samples.	Focuses on previous model errors.
Reduces variance.	Can reduce bias and variance.
Models can train in parallel.	Models generally depend on previous models.
Example: Random Forest.	Example: AdaBoost, XGBoost.

Source: https://www.geeksforgeeks.org/machine-learning/decision-tree-introduction-example/

Knn:

The main idea behind KNN is to find the k-nearest data points to a given test data point and use these nearest neighbors to make a prediction. The value of k is a hyperparameter that needs to be tuned, and it represents the number of neighbors to consider.

- **Lazy learning algorithm** – KNN is a lazy learning algorithm because it does not have a specialized training phase and uses all the data for training while classification.
- **Non-parametric learning algorithm** – KNN is also a non-parametric learning algorithm because it doesn't assume anything about the underlying data.

> Now, we need to classify new data point with black dot (at point 60,60) into blue or red class. We are assuming K = 3 i.e. it would find three nearest data points. It is shown in the next diagram –
>
> 
>
>
> We can see in the above diagram the three nearest neighbors of the data point with black dot. Among those three, two of them lies in Red class hence the black dot will also be assigned in red class.

Source:
https://www.tutorialspoint.com/machine_learning/machine_learning_knn_nearest_neighbors.htm

Naive Bayes Classifiers

Naive Bayes is a machine learning classification algorithm that predicts the category of a data point using probability. It assumes that all features are independent of each other. Naive Bayes performs well in many real-world applications such as spam filtering, document categorization and sentiment analysis.

It is named as "Naive" because it assumes the presence of one feature does not affect other features. The "Bayes" part of the name refers to its basis in Bayes' Theorem.

Assumption of Naive Bayes

The fundamental Naive Bayes assumption is that each feature makes an:

- **Feature independence:** This means that when we are trying to classify something, we assume that each feature (or piece of information) in the data does not affect any other feature.
- **Continuous features are normally distributed**
- **Discrete features have multinomial distributions**
- **Features are equally important**
- **No missing data**

Introduction to Bayes' Theorem

Bayes' Theorem provides a principled way to reverse conditional probabilities. It is defined as:

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

Types of Naive Bayes Model

There are three types of Naive Bayes Model :

1. Gaussian Naive Bayes

In Gaussian Naive Bayes, continuous values associated with each feature are assumed to be distributed according to a Gaussian distribution. A Gaussian distribution is also called Normal distribution When plotted, it gives a bell shaped curve which is symmetric about the mean of the feature values as shown below:

2. Multinomial Naive Bayes

Multinomial Naive Bayes is used when features represent the frequency of terms (such as word counts) in a document. It is commonly applied in text classification, where term frequencies are important.

3. Bernoulli Naive Bayes

Bernoulli Naive Bayes deals with binary features, where each feature indicates whether a word appears or not in a document. It is suited for scenarios where the presence or absence of terms is more relevant than their frequency. Both models are widely used in document classification tasks.

Applications

- **Spam Email Filtering**: Classifies emails as spam or non-spam based on features.

- **Text Classification**: Used in sentiment analysis, document categorization and topic classification.
- **Medical Diagnosis:** Helps in predicting the likelihood of a disease based on symptoms.
- **Credit Scoring:** Evaluates creditworthiness of individuals for loan approval.
- **Weather Prediction**: Classifies weather conditions based on various factors

Source: https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/

Train-Test Split:

The train/test split is a data-splitting technique used to evaluate how well a machine learning model generalizes to new, unseen data. By dividing your dataset into separate subsets, you prevent your model from "memorizing" the answers (overfitting) and ensure that its performance metrics accurately reflect its real-world utility.

- Training Set (Typically 70% – 80%)
- Test Set (Typically 20% – 30%)

Limitations of Train-Test Split:

The result depends heavily on how the data is split. What if the test set is too easy or too hard by chance?

Here **cross validation** comes in.

Cross validation:

Cross-validation is a technique to ensure the model works well on different subsets of the data. It reduces the risk of overfitting and gives more reliable performance estimates.

Instead of relying on a single train-test split (which may be biased), CV allows us to use **multiple train-test splits** systematically.

It's like giving your model multiple mini-exams rather than one final test.

When you split your data into a training set and test set, there's a risk that:

- The split is not representative of the full dataset.
- You may overfit or underfit based on that one split.
- Your model performance is dependent on chance.

Cross-validation solves this by:

- Using all data points for both training and validation.
- Giving more stable performance estimates.

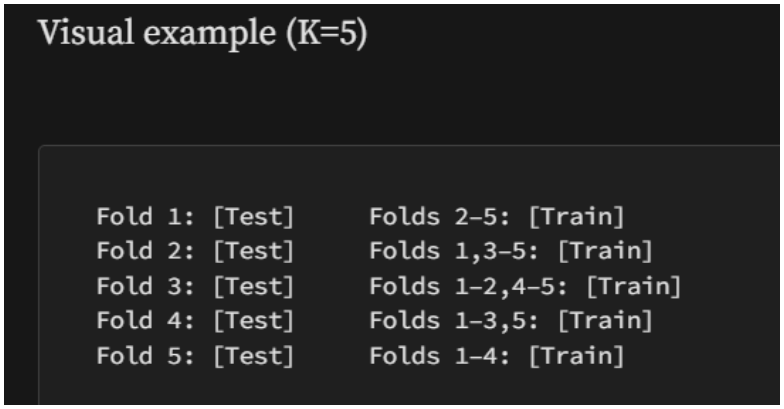
Cross validation can be classified into four types:

1. K-Fold Cross-Validation (Most Common)
2. Stratified K-Fold Cross-Validation
3. Leave-One-Out Cross-Validation (LOOCV)
4. Repeated K-Fold Cross-Validation

The most common type that used for cross validation is K-Fold Cross-Validation.

How it works:

- The dataset is divided into K equal folds (e.g., 5 or 10).
- The model is trained on K-1 folds and tested on the remaining fold.
- This process is repeated K times, each time with a different test fold.
- The results are averaged for a more reliable metric.



In Brief, you can use train test split when the dataset is large and you need quick testing whereas you can use cross validation when the dataset is small and you want reliable evaluation.

Source: https://medium.com/@www.arpitgupta64/train-test-split-cross-validation-in-machine-learning-75f218ffcb38

Feature scaling :

Feature scaling is a method used to normalize the range of independent variables or features of data. In data processing, it is also known as data normalization and is generally performed during the data pre-processing step.

Reason why feature scaling is applied is that gradient descent converges much faster with feature scaling than without it. The goal of applying feature Scaling is to make sure features are on almost the same scale so that each feature is equally important and make it easier to process by most ML algorithms. In my views, any algorithm that computes distance or assumes normality, scale your features.

Source: https://www.kaggle.com/discussions/general/190844

Handling Missing Data in Machine Learning:

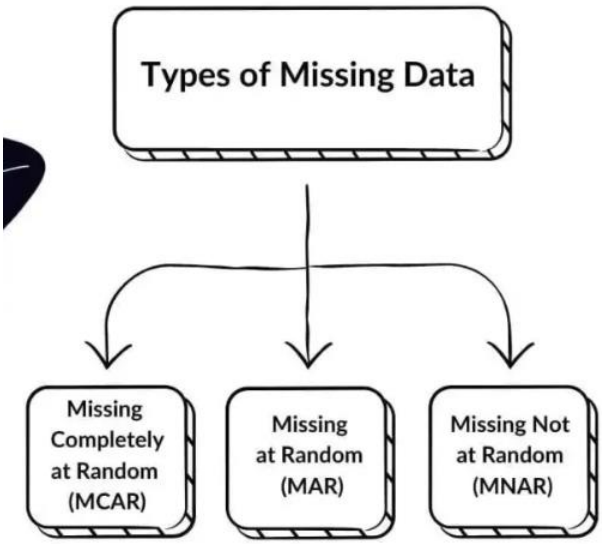
Imagine you want to calculate the average age of all five students. One student's age is missing.

You don't have complete information, so your analysis might become inaccurate or incomplete.

In Machine Learning, missing values can cause problems because many ML algorithms cannot directly process NaN values.

Therefore, before training a model, we usually need to handle missing data.

Types of Missing Data



These three types are extremely important because they explain WHY data is missing.

MCAR – Missing Completely At Random

Random missingness

Meaning: Data is missing for a completely random reason. There is no relationship between the missing value and any other information in the dataset.

Imagine:

You are conducting a survey of 100 students.

You ask students to enter their ages in a form.

Unfortunately, due to a technical problem, the age field randomly disappears for 10 students.

You are collecting customer information from a shopping website. Suddenly, the internet connection disconnects while some customers are submitting forms. Some entries are lost randomly.

This is MCAR.

How can we identify it?

We investigate whether the probability of a value being missing is unrelated to both observed and unobserved data.

How do we handle MCAR?

- Mean imputation
- Median imputation
- Mode imputation
- Removing rows with missing values (when only a small proportion of data is missing)

MAR – Missing At Random

Missing because of another observed variable

Meaning: Data is missing because of some other information that we already have in our dataset.

Don't confuse this with MCAR. In MAR, there is a reason behind missing data, but that reason is related to another observed variable.

Let's understand with an example.

Imagine you are collecting information about students.

You have two columns:

- Age
- Education Level

Your dataset:

Student	Age	Education Level
Ali	20	Undergraduate
Sara	NaN	School
Ahmed	22	Undergraduate

Ayesha	NaN	School
Zain	24	Undergraduate

Suppose you discover that younger students are less likely to enter their ages in the form because they are completing the survey through a school account with a faulty form.

You already have their education level, so you can observe a pattern:

- School students have more missing ages.
- Undergraduate students have fewer missing ages.

The missingness is related to Education Level, which is already available in the dataset.

This is MAR.

How do we handle MAR?

- Regression imputation
- KNN imputation
- MICE (Multivariate Imputation by Chained Equations)
- Iterative Imputation

MICE stands for Multivariate Imputation by Chained Equations. It assumes data is Missing At Random (MAR).

- Fill all NaNs with a placeholder (eg. mean)
- Select one column (eg. Age) back to missing.
- Treat "Age" as the target variable and build a regression model (eg. Linear Regression) using other columns (Fare, Pclass) as features to predict Age.
- Repeat for all columns in a cycle (round-robin) until convergence.

Scikit-Learn provides this as IterativeImputer.

MNAR – Missing Not At Random

Missing because of the missing value itself or unobserved information

Meaning: Data is missing because of the actual value that is missing, or some other factor that we have not observed.

Example:

Imagine a hospital collecting patients' weights.

Patient	Weight	Health Condition
Ali	65 kg	Normal
Sara	NaN	Overweight
Ahmed	70 kg	Normal
Ayesha	NaN	Overweight
Zain	60 kg	Normal

Suppose overweight patients are more likely to hide their actual weight because they feel uncomfortable sharing it.

Notice something important:

- Their weight is missing.
- Their weight itself influences whether they provide it.
- We cannot fully understand the missingness just by looking at other available columns.

This is MNAR.

How do we handle MNAR?

- Collect additional information about why values are missing.
- Use domain knowledge and statistical models that explicitly account for missingness.
- Perform sensitivity analysis.
- Consider selection models or pattern-mixture models.

Simple mean imputation or ordinary MICE does not automatically solve MNAR.

Encoding Categorical Features

Understanding categorical features

Imagine you are working on a Machine Learning project for a shopping website.

You have collected information about customers:

Customer	Age	Gender	City	Payment Method	Purchase Amount
Ali	22	Male	Sukkur	Cash	5000

Sara	25	Female	Karachi	Card	7000
Ahmed	30	Male	Lahore	Online	3000
Ayesha	21	Female	Sukkur	Cash	6000

Some columns contain numbers:

- Age
- Purchase Amount

Other columns contain categories or labels:

- Gender
- City
- Payment Method

Categorical features are columns that represent different groups, labels, types, or categories instead of numerical measurements.

Why do we need Encoding?

Many traditional ML algorithms work with numerical input values. They cannot directly understand text categories such as Male, Female, Lahore.

So, we need to convert these categories into numerical representations.

This process is called Encoding Categorical Features.

Encoding does not mean changing the actual meaning of the data. It means representing categories in a format that a model can process.

Types of Categorical Features

- **Nominal data** means categories that do not have any particular ranking or order.
- **Ordinal data** means categories that have a natural ranking or order.

Types of Encoding Techniques

1. **Label Encoding:** assigns a unique integer to every category.
2. **Ordinal Encoding:** is used when categories have a meaningful order.
3. **One-Hot Encoding:** is one of the most widely used encoding techniques. It converts each category into a separate binary column containing 0 or 1.

4. **Binary Encoding:** is another technique for converting categorical values into numerical representations.
5. **Frequency Encoding:** replaces each category with the number of times it appears in the dataset.
6. **Target Encoding:** replaces categories with a statistic calculated from the target variable, often the target mean for a regression or binary classification problem.

Feature Selection vs Dimensionality Reduction

Both are techniques used in Machine Learning to reduce the number of features in a dataset, improve model performance, and reduce computational complexity. However, they work differently.

1. Feature Selection

Feature Selection is the process of selecting the most relevant features from an existing dataset while removing unnecessary or less important features.

- It keeps the original features unchanged.
- It removes irrelevant, redundant, or less useful features.
- It helps reduce overfitting and improves model efficiency.

Example:

Suppose we have a dataset containing:

Age, Salary, Education, Height, Weight

If we are predicting salary, we might select only:

Age, Education

The remaining features are excluded because they may not contribute meaningfully to the prediction.

Dimensionality Reduction

Dimensionality Reduction is the process of reducing the number of features by transforming the original features into a smaller set of new features while preserving as much important information as possible.

- It creates new features from existing ones.
- It may combine multiple features into fewer dimensions.

- It helps simplify complex datasets and improve computational efficiency.

Example:

Suppose a dataset contains 10 features. Using PCA (Principal Component Analysis), we can transform these 10 features into 2 new components that retain most of the important information.